

Task 18: Unit Testing for Microservices

AIM

To write unit test cases for microservices using a testing framework (Jest), testing service logic, REST controllers, and data handling independently to ensure reliability.

ALGORITHM

- Step 1: Set up Jest testing framework with supertest for HTTP testing.
- Step 2: Write unit tests for service logic (business functions).
- Step 3: Write integration tests for REST controllers using supertest.
- Step 4: Write tests for data handling and validation.
- Step 5: Mock external dependencies to isolate units under test.
- Step 6: Run all tests and verify pass/fail results with coverage report.

PROGRAM

```
// ----- productService.js (Service Logic) -----  
  
const products = [  
  { id: 1, name: 'Laptop', price: 75000 },  
  { id: 2, name: 'Mouse', price: 500 },  
];  
  
const getAllProducts = () => products;  
const getProductById = (id) => products.find(p => p.id === id) || null;  
const addProduct = (product) => {  
  if (!product.name || !product.price) throw new Error('Name and price are required');  
  if (product.price < 0) throw new Error('Price must be positive');  
  const newProduct = { id: products.length + 1, ...product };  
  products.push(newProduct);  
  return newProduct;  
};  
  
module.exports = { getAllProducts, getProductById, addProduct };  
  
// ----- app.js (REST Controller) -----
```

```

const express = require('express');
const { getAllProducts, getProductById, addProduct } = require('./productService');
const app = express();
app.use(express.json());

app.get('/api/products', (req, res) => res.json(getAllProducts()));
app.get('/api/products/:id', (req, res) => {
  const product = getProductById(parseInt(req.params.id));
  if (!product) return res.status(404).json({ error: 'Not found' });
  res.json(product);
});
app.post('/api/products', (req, res) => {
  try {
    const product = addProduct(req.body);
    res.status(201).json(product);
  } catch (e) { res.status(400).json({ error: e.message }); }
});
module.exports = app;

// ----- tests/productService.test.js -----
const { getAllProducts, getProductById, addProduct } = require('../productService');

describe('Product Service Logic', () => {
  test('getAllProducts returns array of products', () => {
    const products = getAllProducts();
    expect(Array.isArray(products)).toBe(true);
    expect(products.length).toBeGreaterThan(0);
  });

  test('getProductById returns correct product', () => {
    const product = getProductById(1);
    expect(product).toBeDefined();
    expect(product.name).toBe('Laptop');
  });

  test('getProductById returns null for invalid id', () => {
    expect(getProductById(999)).toBeNull();
  });
});

```

```

test('addProduct throws error without name', () => {
  expect(() => addProduct({ price: 100 })).toThrow('Name and price are required');
});

test('addProduct throws error for negative price', () => {
  expect(() => addProduct({ name: 'Test', price: -10 })).toThrow('Price must be positive');
});

// ----- tests/controller.test.js -----

const request = require('supertest');
const app = require('../app');

describe('REST Controller Tests', () => {
  test('GET /api/products returns 200 with products', async () => {
    const res = await request(app).get('/api/products');
    expect(res.status).toBe(200);
    expect(Array.isArray(res.body)).toBe(true);
  });

  test('GET /api/products/1 returns product', async () => {
    const res = await request(app).get('/api/products/1');
    expect(res.status).toBe(200);
    expect(res.body.name).toBe('Laptop');
  });

  test('GET /api/products/999 returns 404', async () => {
    const res = await request(app).get('/api/products/999');
    expect(res.status).toBe(404);
  });

  test('POST /api/products creates product', async () => {
    const res = await request(app).post('/api/products')
      .send({ name: 'Monitor', price: 20000 });
    expect(res.status).toBe(201);
    expect(res.body.name).toBe('Monitor');
  });

  test('POST /api/products with invalid data returns 400', async () => {

```

```
const res = await request(app).post('/api/products').send({ name: '' });  
expect(res.status).toBe(400);  
});  
});
```

OUTPUT

Test Execution Output:

\$ npx jest --verbose

PASS tests/productService.test.js

Product Service Logic

- ✓ getAllProducts returns array of products (2 ms)
- ✓ getProductById returns correct product (1 ms)
- ✓ getProductById returns null for invalid id (1 ms)
- ✓ addProduct throws error without name (1 ms)
- ✓ addProduct throws error for negative price (1 ms)

PASS tests/controller.test.js

REST Controller Tests

- ✓ GET /api/products returns 200 with products (15 ms)
- ✓ GET /api/products/1 returns product (5 ms)
- ✓ GET /api/products/999 returns 404 (4 ms)
- ✓ POST /api/products creates product (6 ms)
- ✓ POST /api/products with invalid data returns 400 (3 ms)

Test Suites: 2 passed, 2 total

Tests: 10 passed, 10 total

Time: 1.245 s

RESULT

Successfully wrote and executed unit tests for microservices covering service logic, REST controllers, and data validation. All 10 test cases passed, ensuring reliability of the application.